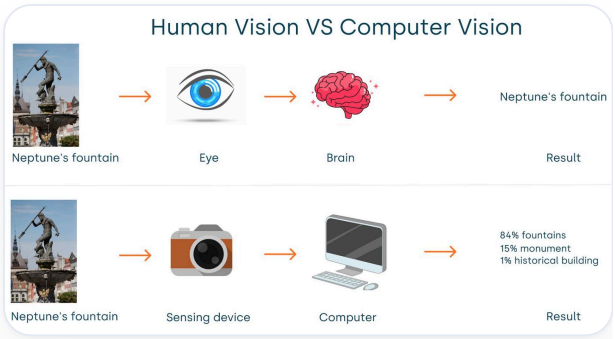
# Enseñando a las máquinas a ver

La visión artificial es la disciplina de la inteligencia artificial que busca capacitar a las máquinas para **extraer información significativa y comprender el mundo a partir de imágenes y vídeos digitales**, imitando la percepción humana.




## La brecha semántica: ¿cómo lo ve un humano vs. una máquina?

### Visión humana

Inmediatamente identificamos «un gato naranja durmiendo en un sofá» y comprendemos su contexto.

### Visión computacional

Una máquina percibe una compleja **matriz de millones de píxeles**, cada uno con valores numéricos RGB como [255, 128, 64…].

Las imágenes digitales son matrices de píxeles. La visión artificial aprende representaciones que permiten asociar esos valores con objetos, relaciones o regiones.

# Tareas principales en visión artificial

### Clasificación

Asigna una etiqueta a la imagen entera: dice qué hay, no dónde.

### Detección de objetos

Encierra cada objeto en un recuadro delimitador y le asigna una clase.

### Segmentación

Define el contorno exacto de cada objeto a nivel de píxel.

### Detección de anomalías

Identifica eventos o datos inusuales que no coinciden con un patrón esperado.

### Estimación de pose

Localiza puntos clave (articulaciones) en personas o animales para comprender su postura.

### Otras

OCR, estimación de profundidad, superresolución, keypoint matching…



# ¿Qué salida necesitas?

La misma imagen admite tareas distintas.

### Quieres contar coches y localizar cada uno.

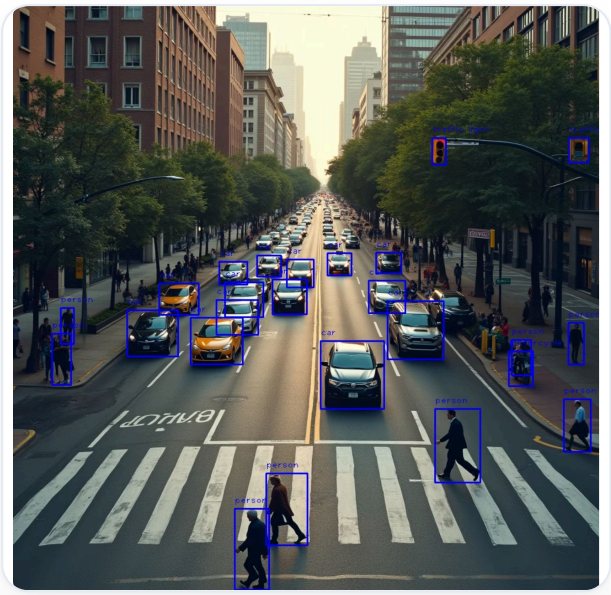
- A Una etiqueta para toda la imagen.
- B Una caja y una categoría por objeto.
- C Una puntuación de confianza para la imagen.

Respuesta B. Exacto: la detección localiza cada instancia y permite contarla.

La clasificación da una etiqueta global. La detección devuelve cajas y categorías por instancia. La segmentación delimita píxeles. Otros problemas incluyen anomalías, pose, OCR, profundidad y superresolución.

# 1. Detección de objetos

La detección de objetos va más allá de la simple clasificación. Su **objetivo principal es localizar y clasificar múltiples objetos simultáneamente** dentro de una imagen, proporcionando para cada uno una caja delimitadora, su categoría y una puntuación de confianza.

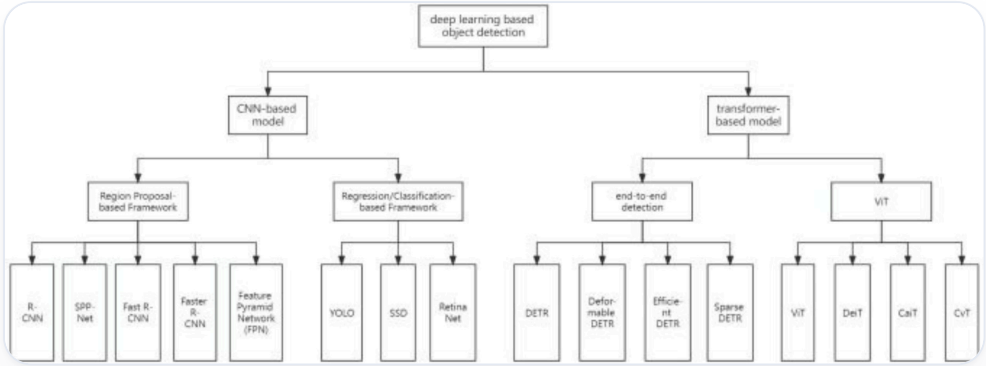


**En qué se diferencia de las otras tareas:**

- **Clasificación:** identifica qué hay en la imagen (una etiqueta global).
- **Detección:** localiza cada objeto con cajas y etiquetas múltiples.
- **Segmentación:** delimita el contorno exacto píxel a píxel.
- **Detección de anomalías:** indica si algo en la imagen se sale de lo normal.

# Tipos de arquitecturas de Object Detection

Hay muchas arquitecturas de detección. Estas son las familias principales.




La rama basada en CNN se divide en frameworks con propuestas de región (R-CNN, SPP-Net, Fast R-CNN, Faster R-CNN, FPN) y frameworks de regresión o clasificación directa (YOLO, SSD, RetinaNet). La rama basada en transformer agrupa detección extremo a extremo (DETR, Deformable DETR, Efficient DETR, Sparse DETR) y arquitecturas ViT (ViT, DeiT, CaiT, CvT).

# Evolución histórica de los detectores

Los modelos de detección de objetos han cambiado mucho en la última década. Nos centraremos en tres grandes etapas arquitectónicas.

1. **Two-Stage (2014-2016)**

   R-CNN, Fast R-CNN, Faster R-CNN: arquitecturas de dos etapas que primero proponen regiones candidatas y luego las clasifican. Precisos pero complejos y lentos.

2. **One-Stage (2015 →)**

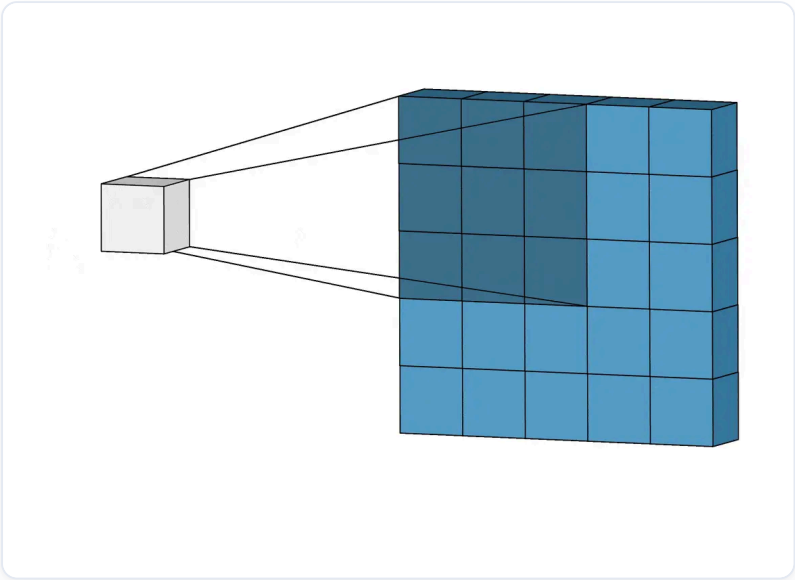
   YOLO cambia el planteamiento: una sola red neuronal procesa la imagen completa en tiempo real, prediciendo cajas y clases simultáneamente mediante una cuadrícula.

3. **Transformers (2020 →)**

   DETR y RF-DETR: pipelines limpios basados en atención, sin anchors predefinidos ni supresión de no máximos (NMS). Arquitectura más simple y generalizable.

# Redes neuronales convolucionales (CNN)

Las redes neuronales convolucionales (CNN) son la base de casi todos los detectores anteriores a 2020 y siguen dentro de muchos actuales. Procesan la imagen directamente con capas que detectan, de forma jerárquica, bordes, texturas y patrones cada vez más complejos.

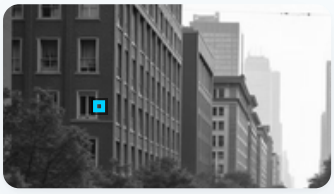


Al apilar capas, la red pasa de detectar bordes a reconocer partes de objetos y objetos enteros, sin que nadie programe esas características a mano.

Las CNN incorporan localidad e invariancia aproximada a traslaciones. Siguen dentro de muchos detectores actuales, y una CNN también capta contexto global si se le añaden capas de atención o un campo receptivo grande.

# Lo que hace una capa convolucional

Un filtro de 3 × 3 recorre la imagen: cada píxel de salida es la suma de sus nueve vecinos multiplicados por nueve pesos. Cambia el filtro y mira qué se enciende; toca o pasa el ratón por las imágenes para ver la cuenta.



Entrada, 192 × 108 píxeles



Mapa de características

| Vecindario | | | | Pesos | | | | Salida |
|---|---|---|---|---|---|---|---|---|
| 17 | 47 | 122 | | −1 | 0 | 1 | | |
| 18 | 51 | 124 | × | −2 | 0 | 2 | = | **419** |
| 20 | 51 | 122 | | −1 | 0 | 1 | | |

Se encienden los cambios de luz de izquierda a derecha: las aristas verticales de los edificios. El cielo, liso, queda negro.

Filtros escritos a mano y calculados en tu navegador. Una CNN aprende sus pesos durante el entrenamiento, y los de su primera capa acaban pareciéndose a estos.

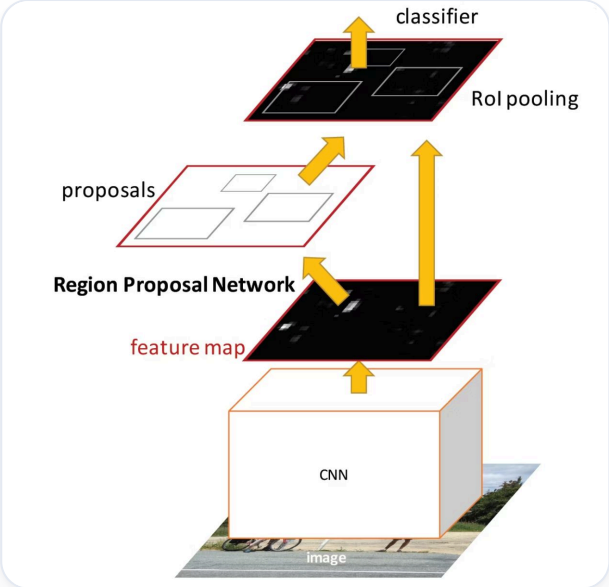
Esta página recoge el filtro de bordes verticales: pesos negativos a la izquierda y positivos a la derecha, de modo que la suma es grande donde la luz cambia en horizontal y cero en las zonas lisas. En el punto marcado, sobre una arista del edificio de la izquierda, la salida vale 419. El filtro de bordes horizontales es el mismo girado 90 grados y enciende cornisas y filas de ventanas. El desenfoque promedia los vecinos y el realce resta los vecinos al centro. Una capa convolucional aplica decenas de filtros como estos a la vez, y las capas siguientes combinan sus mapas para detectar texturas, partes y objetos.

# Two-Stage: precisión a costa de complejidad

Los detectores de dos etapas, como la familia R-CNN, dividen el problema en **dos fases secuenciales**: primero generan propuestas de regiones candidatas (lugares donde podría haber objetos), y luego clasifican y refinan cada propuesta.

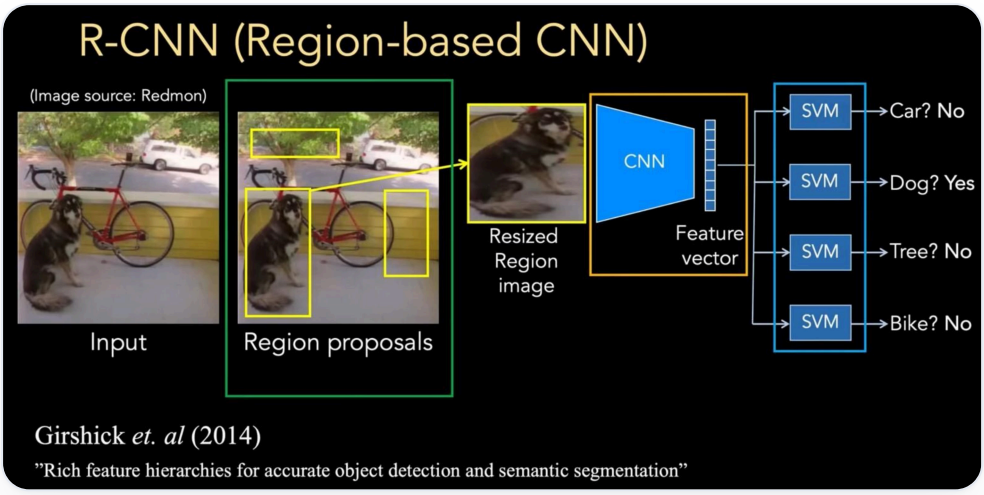
**ROI Pooling** acelera el proceso al extraer features de forma eficiente, pero el pipeline sigue siendo computacionalmente costoso. La generación inicial de regiones puede proponer cientos de candidatos, ralentizando la inferencia.

**Trade-off característico:** excelente precisión en benchmarks, pero latencia elevada para aplicaciones en tiempo real. Ideales cuando la exactitud es más importante que la velocidad. Difícil de optimizar.



# R-CNN

Region proposal + CNN + clasificador.




Girshick et al. (2014), «Rich feature hierarchies for accurate object detection and semantic segmentation».

# Comparación de velocidad de RCNNs

**Results**

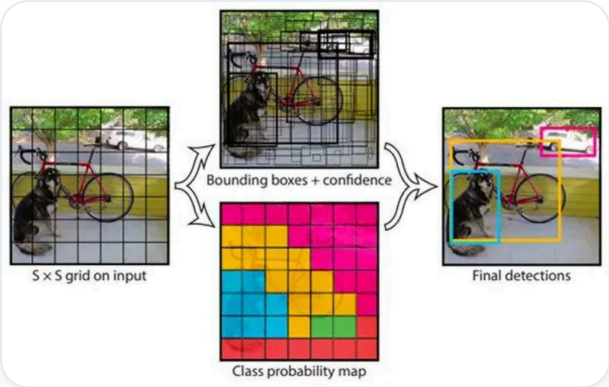
| | RCNN | Fast RCNN | Faster RCNN |
|---|---|---|---|
| **Proposal Generation Method** | Selective Search | Selective Search | RPN |
| **Proposal Generation Time** | 1500 ms | 1500 ms | 10 ms |
| **Predictions for Generated Proposals Time / Image** | 47000 ms | 320 ms | 190 ms |
| **Speedup Factor** | X | 25X | 250X |

El salto está en la generación de propuestas: Selective Search tarda 1500 ms y la RPN de Faster R-CNN, 10 ms.

La tabla compara tres generaciones. Método de propuestas: Selective Search en RCNN y Fast RCNN, RPN en Faster RCNN. Tiempo de generación: 1500 ms, 1500 ms y 10 ms. Predicción sobre las propuestas por imagen: 47000 ms, 320 ms y 190 ms. Factor de aceleración: 1×, 25× y 250×.

# YOLO: detección como regresión directa

YOLO (You Only Look Once) reformula la detección como un **problema de regresión único**. La imagen se divide en una cuadrícula S×S, donde cada celda predice directamente las cajas delimitadoras y las probabilidades de clase de los objetos cuyo centro cae en ella.




**La ventaja:** una única pasada hacia adelante por la red permite velocidades de procesamiento en tiempo real (30+ FPS), manteniendo precisión competitiva. Esta arquitectura one-stage elimina la necesidad de generar propuestas de región por separado.

- Cada celda predice múltiples cajas con sus scores de confianza.
- Las predicciones ocurren simultáneamente para toda la imagen.
- Ideal para aplicaciones que requieren latencia mínima.

# La arquitectura YOLO: velocidad y simplicidad

Cómo el modelo procesa la imagen en una sola pasada («You Only Look Once»).

1 **La estrategia de la cuadrícula (S×S Grid)**

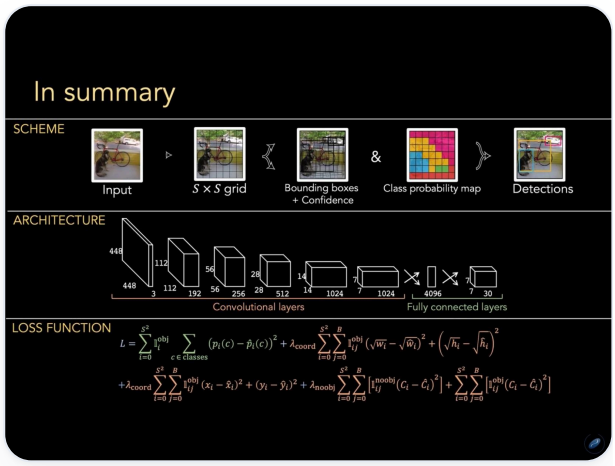
YOLO divide la imagen de entrada en una cuadrícula. Si el centro de un objeto cae en una celda específica, esa celda es la responsable de detectarlo.

2 **Predicción simultánea (bounding boxes + clases)**

Cada celda predice recuadros (bounding boxes) con su ubicación, tamaño y nivel de confianza, y la categoría del objeto (mapa de probabilidad), todo al mismo tiempo.

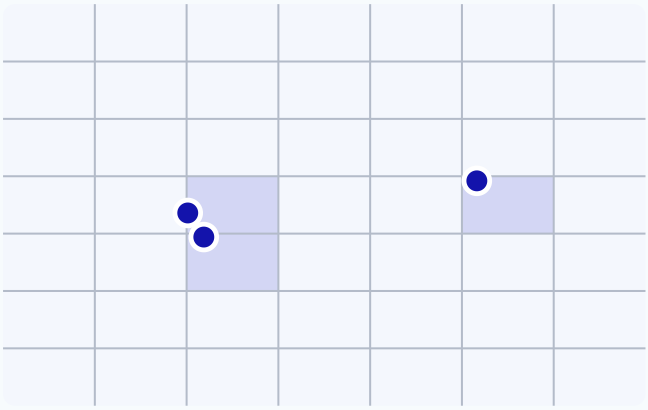
3 **Arquitectura unificada (CNN)**

Una única Red Neuronal Convolucional (CNN) procesa la imagen completa, conectando directamente los píxeles de entrada con las coordenadas y clases de salida, eliminando etapas intermedias complejas.



# Una celda se responsabiliza de un objeto

Baja la cuadrícula y mira qué pasa cuando dos centros caen en la misma celda.



Cada punto es el centro de un objeto. YOLOv1 usaba una cuadrícula de 7 × 7.

Objetos con celda propia

## 3 de 3

Cada objeto cae en una celda distinta: la cuadrícula puede hacerse cargo de todos.

Esta página recoge el estado con una cuadrícula de 7 × 7, la que usaba YOLOv1. Al bajar a 6 × 6 o menos, los centros de los dos coches de la izquierda caen en la misma celda y esa celda solo puede responsabilizarse de uno: es la limitación más citada de YOLOv1 con objetos juntos o pequeños. Subir S reparte mejor los centros en general, pero no lo garantiza: con 8 × 8 y con 12 × 12 esos dos centros vuelven a caer juntos, porque lo que decide es dónde quedan las líneas de la rejilla. Además, más celdas multiplican las predicciones y el coste. Las versiones posteriores atacan el problema con predicción multiescala y, más tarde, con enfoques anchor-free.

# Una sola pasada, una sola red

Comprueba en qué se diferencia un detector de una etapa.

## ¿Qué describe la idea de un detector de una etapa?

- A Propone regiones y las clasifica en un segundo recorrido.
- B No necesita imágenes de entrenamiento.
- C Predice localización y categoría en una sola pasada de la red.

Respuesta C. Exacto: esa es la idea central, aunque los detalles cambian entre versiones.

Un detector de una etapa predice cajas y clases en una única pasada sobre toda la imagen, sin una fase previa que proponga regiones candidatas.

# Evolución de la familia YOLO (2016-2026)

Diez años de versiones publicadas por grupos distintos. El número no indica quién hizo cada modelo ni que herede el código del anterior.

### 2016 · YOLOv1

Detección en tiempo real con una sola pasada de la red. Mucho más rápido que los detectores de dos etapas, aunque menos preciso.

### 2018 · YOLOv3

Predicción a tres escalas y backbone Darknet-53, con mejor detección de objetos pequeños.

### 2020 · YOLOv4

Backbone CSPDarknet53 y trucos de entrenamiento («Bag of Freebies», como el aumento Mosaic) que mejoran la precisión sin frenar la inferencia.

### 2020 · YOLOv5

Primera versión en PyTorch, de Ultralytics. Popularizó YOLO por lo fácil que era entrenarlo y exportarlo.

### 2022 · YOLOv7

Arquitectura E-ELAN. Superó en velocidad y precisión a los detectores conocidos entre 5 y 160 FPS.

### 2023 · YOLOv8

De Ultralytics. Pasó a una cabeza sin anchors y unificó en una API detección, segmentación, pose y clasificación.

### 2024 · YOLOv9

PGI y GELAN para que la información útil no se pierda en redes profundas. La rama auxiliar se quita al inferir y no añade coste.

### 2024 · YOLOv10

De la Universidad de Tsinghua. Añade una asignación uno a uno al entrenar y prescinde del NMS al inferir.

### 2024 · YOLO11

Sucesor de YOLOv8 en Ultralytics, sin la «v» en el nombre: más precisión con menos parámetros.

### 2025 · YOLO12

Versión comunitaria centrada en la atención: introduce atención por áreas sin perder el tiempo real.

### 2026 · YOLO26

Última de Ultralytics (enero). Cabeza uno a uno opcional sin NMS y regresión sin DFL, pensada para edge.

# Por qué las CNN no son suficientes

Las CNN cambiaron la visión artificial, pero tienen límites de arquitectura que se notan cuando importan los objetos alejados entre sí o las relaciones espaciales que no son evidentes.

### Campo receptivo limitado

Las CNN procesan información en parches locales. Para entender relaciones a larga distancia, necesitan muchas capas, lo que aumenta la profundidad del modelo y dificulta la propagación de gradientes. Esto es ineficiente para capturar el contexto global de una imagen.

### Componentes ad hoc

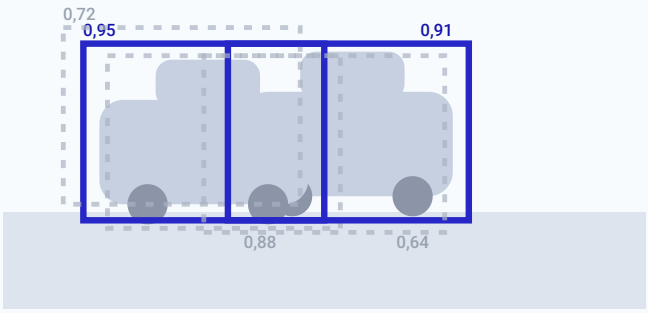
Los detectores basados en CNN dependen de piezas manuales y heurísticas, como los «anchors» predefinidos (cajas de distintos tamaños y proporciones) y la supresión de no máximos (NMS) para eliminar duplicados. Estas piezas añaden complejidad e hiperparámetros, y no se optimizan junto con el resto de la red.

### Sesgo inductivo fuerte

Asumen una fuerte localidad espacial e invarianza a traslación. Esto es útil para objetos que aparecen en cualquier lugar de la imagen, pero dificulta la modelización de relaciones abstractas o estructuras que no se ajustan a patrones de vecindad directa.

# El umbral de NMS es un hiperparámetro

El NMS mide cuánto se solapan dos cajas (su IoU, que se define unas pantallas más adelante). Baja el umbral y desaparece un coche entero; súbelo y vuelven los duplicados.




Cinco cajas candidatas sobre dos coches. El detector las propone todas antes de filtrar.

Cajas que quedan

**2**

Suprimidas

**3**

Una caja por coche: el umbral limpia los duplicados sin borrar objetos.

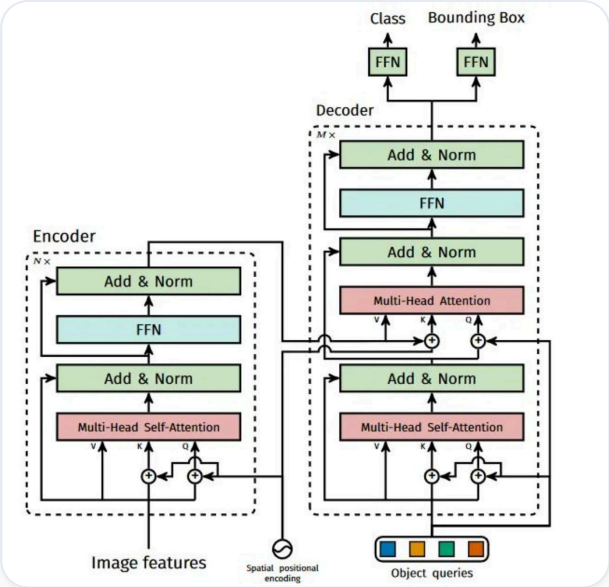
Esta página recoge el estado con umbral 0,50. El algoritmo recorre las cajas de mayor a menor puntuación: cada superviviente suprime a las que solapan con ella por encima del umbral. Con 0,20 la caja del primer coche llega a suprimir la del segundo, porque comparten un 25 % de área, y se pierde un objeto real. Con 0,80 apenas se suprime nada y quedan las cinco cajas. No hay un valor universalmente correcto: depende de cuánto se solapen los objetos del dominio, y por eso DETR y RF-DETR eliminan este paso mediante la asignación uno a uno durante el entrenamiento.

# DETR: Transformers para detección

DETR (DEtection TRansformer, Carion et al., 2020) trata la detección como la **predicción directa de un conjunto**: una arquitectura encoder-decoder con «object queries» aprendibles devuelve de una vez un número fijo de predicciones.

**Pipeline simplificado:** un backbone (como ResNet) extrae features visuales, el encoder Transformer las procesa globalmente, y el decoder utiliza N queries (típicamente 100) para predecir N potenciales objetos, cada uno con su caja y clase.

- Sin hiperparámetros de NMS que ajustar.
- Sin anchors dependientes del dataset.
- Se entrena de extremo a extremo, con todo el pipeline diferenciable.



# Fin del NMS: matching húngaro

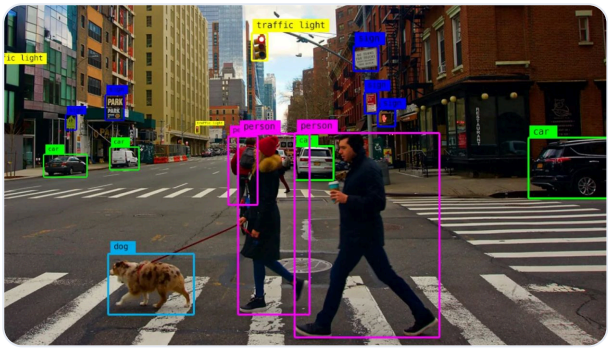
DETR elimina la supresión de no máximos (NMS) con el matching húngaro, un algoritmo de asignación bipartita. Durante el entrenamiento empareja cada objeto real con una sola predicción, la de menor coste global. Como cada objeto tiene una única predicción responsable, el modelo aprende a no duplicar y el filtrado final deja de hacer falta.

### YOLO antes del NMS: predicciones redundantes




Predice miles de cajas candidatas y necesita un proceso de filtrado (NMS) para eliminar las duplicadas.

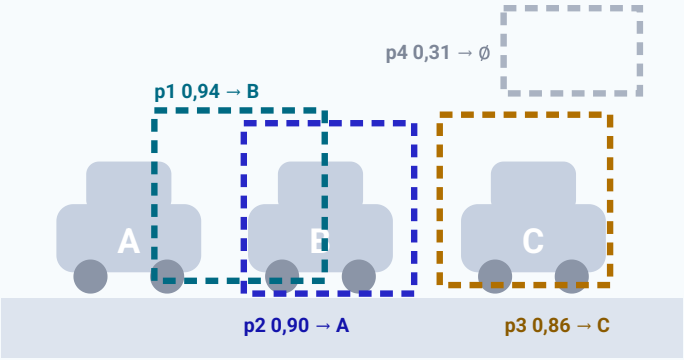
### DETR: predicciones directas y únicas




Asigna una predicción única a cada objeto real utilizando un algoritmo de matching húngaro basado en un coste global.

# Emparejar de una en una no basta

Mueve p1, la predicción más segura, hacia el coche B. Emparejar por orden de confianza le da B y deja a p2 sin pareja útil; la asignación húngara minimiza el coste total.




Coste de cada pareja: 1 − IoU

| | A | B | C |
|---|---|---|---|
| p1 | 0,80 | **0,67** | 1,00 |
| p2 | **1,00** | 0,14 | 1,00 |
| p3 | 1,00 | 1,00 | **0,14** |
| p4 | 1,00 | 1,00 | 1,00 |

Coste por confianza

**1,81**

Coste húngaro

**1,08**

p1 elige primero y se queda con B, que es el que más solapa. p2 tiene que conformarse con A, al que no toca: el coste total sube a 1,81.

Coste simplificado: solo cuenta el solape. DETR suma además el error de clase y la distancia L1 entre cajas.

Esta página recoge el estado con p1 en x = 36, emparejando por orden de confianza. p1 solapa un 33 % con B y un 20 % con A, así que elige B; p2, que cubría B casi perfectamente, se queda con A y no lo toca. Coste total: 1,81. La asignación húngara prueba el reparto completo y prefiere p1 con A y p2 con B: 1,08. Con p1 por debajo de x = 34 los dos métodos coinciden. DETR usa la húngara durante el entrenamiento; p4, la predicción sobrante, aprende a decir «ningún objeto».

# Una predicción por objeto

La asignación bipartita ocurre durante el entrenamiento, no en la inferencia.

### ¿Para qué sirve la asignación bipartita en DETR durante el entrenamiento?

- A Relacionar predicciones y objetos reales uno a uno al calcular la pérdida.
- B Aplicar NMS a las cajas de cada imagen al predecir.
- C Crear automáticamente etiquetas para entrenar.

Respuesta A. Exacto: define qué objetivo supervisa cada predicción.

El coste combina clase y geometría. Al forzar una correspondencia uno a uno, el modelo aprende a no duplicar predicciones y deja de necesitar NMS al inferir.

# RF-DETR

RF-DETR (Roboflow, 2025; publicado en ICLR 2026) es un detector en tiempo real de la familia DETR. Su versión 2XL fue el primer detector en tiempo real en pasar de 60 AP en COCO, y la nano da 48,0 AP, 5,3 más que D-FINE nano con una latencia parecida.

### Backbone DINOv2 preentrenado

Parte de un vision transformer entrenado sin etiquetas sobre millones de imágenes (tema 4). Por eso se adapta rápido a un dominio nuevo con pocos datos.

### Atención deformable en el decoder

En lugar de mirar todos los píxeles, cada query consulta unos pocos puntos de muestreo en cada escala. Así trabaja con mapas de alta resolución sin disparar el coste, lo que ayuda con los objetos pequeños.

### Sin anchors ni NMS, en varios tamaños

Como todo DETR, no necesita anchors predefinidos ni supresión de no máximos. Una búsqueda de arquitectura (NAS) entrena una sola red y saca de ella versiones de nano a 2XL sin reentrenar.

# De Deformable DETR a RF-DETR

RF-DETR hereda el decoder de Deformable DETR y cambia la primera mitad: donde aquel usaba una CNN y un encoder con atención deformable, RF-DETR pone DINOv2.

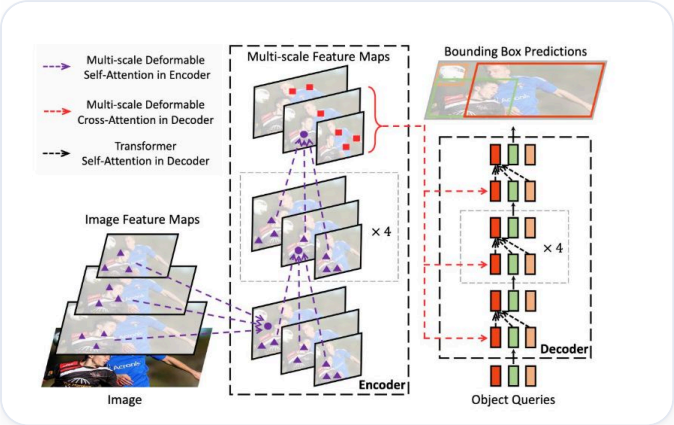



Figura de Deformable DETR (Zhu et al., 2021). RF-DETR conserva el decoder de la derecha y sustituye el encoder.

1

### Features a varias escalas

Deformable DETR toma mapas de varias resoluciones de una CNN. RF-DETR los saca de la salida de DINOv2, con un proyector que la reescala a varios tamaños.

2

### El encoder

En Deformable DETR, un encoder con autoatención deformable (morado en la figura). En RF-DETR, el propio DINOv2, que alterna atención por ventanas y atención global para equilibrar precisión y latencia.

3

### Cross-attention deformable

Las «object queries» consultan las features de todas las escalas en unos pocos puntos cada una (rojo en la figura). Esta parte es igual en los dos modelos.

4

### Refinamiento de predicciones en el decoder

Varias capas de decoder refinan las predicciones: cada una parte de las cajas de la anterior y las corrige.

5

### Salida final de bounding boxes

El modelo entrega una caja y una clase por query, sin el posprocesado de NMS.

# Por qué NO usamos Ultralytics en clase

Ultralytics es muy popular, pero este curso necesita código abierto que se pueda leer por dentro y con licencias que no limiten ni la investigación ni el uso industrial.

### Licencia problemática

Código y pesos van con AGPL-3.0, que obliga a publicar el código de la aplicación que los incorpora cuando la distribuyes, y también cuando solo la ofreces como servicio en red. Para evitarlo hay que pagar su licencia comercial. AGPL es open source, pero choca con la mayoría de usos industriales.

### Apropiación de marca YOLO

Sus versiones (YOLOv5, YOLOv8, YOLO11 y YOLO26) se presentan como la continuación de YOLO, pero no vienen de Joseph Redmon ni de la línea académica que siguió su trabajo. Como otros grupos numeran también sus YOLO (v6, v7, v9, v10, v12), el número ya no dice de dónde sale cada modelo.

### Código no modular

Está pensado para usarse a través de su CLI y de su API de alto nivel. Estudiarlo por dentro o adaptarlo a un proyecto de investigación cuesta bastante más.

### Limitaciones académicas

Varias versiones salieron sin un paper que describa la arquitectura y los experimentos, lo que dificulta reproducir resultados y explicarlos en clase.

Las condiciones de licencia se revisan por versión y por artefacto: el código y los pesos pueden tener términos distintos y cambiar entre versiones. Conviene comprobar la licencia vigente del repositorio y del checkpoint concreto antes de decidir.

# Por qué SÍ usamos RF-DETR en clase

RF-DETR es el detector de referencia del curso: su arquitectura es moderna y legible, y el proyecto es open source con licencia Apache 2.0.

### Licencia Apache 2.0

Permite usarlo, modificarlo y venderlo en industria, investigación y enseñanza, sin obligación de publicar tu código.

### Arquitectura moderna y clara

Es un DETR: backbone, decoder con queries y salida directa, sin anchors ni NMS. Entenderlo es entender cómo se construyen los detectores actuales.

### Código limpio y modular

Se lee y se modifica bien, y el mismo paquete sirve para inferir y para reentrenar con tus datos.

### Velocidad y precisión

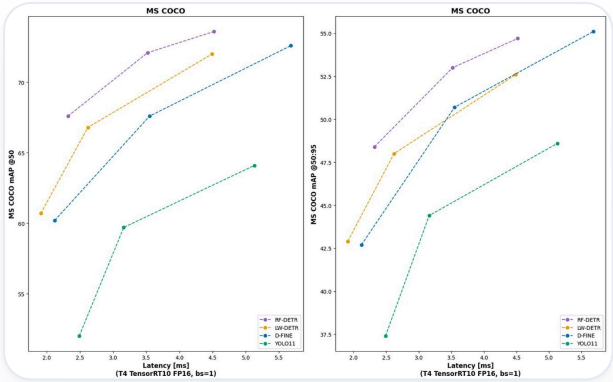
A igual latencia en una T4, da más mAP en COCO que YOLO11 según las curvas de su repositorio (siguiente pantalla).

# RF-DETR vs YOLO en producción

La elección entre RF-DETR y YOLO depende de tus **prioridades específicas de producción**.

**Consideraciones prácticas:**

- **Licencia:** los YOLO de Ultralytics usan AGPL-3.0, que obliga a publicar el código de la aplicación que los incorpora al distribuirla o servirla; RF-DETR usa Apache 2.0, que no lo exige.
- **Mantenibilidad:** RF-DETR evita el ajuste manual de anchors y umbrales de NMS.
- **Adopción:** YOLO tiene más tutoriales, integraciones y formatos de exportación probados, y más gente que ya lo conoce.




Latencia frente a mAP en MS COCO, medida en T4 con TensorRT10 y FP16.

Las curvas proceden del repositorio de RF-DETR y se miden en un hardware concreto. Antes de decidir conviene reproducir la medida en el dispositivo de destino, con el tamaño de entrada, la precisión numérica y el tamaño de lote que se vayan a usar realmente.

# IoU (Intersection over Union)

La **Intersection over Union (IoU)** mide cuánto se solapan dos cajas delimitadoras: la que predice el modelo y la caja de verdad (ground truth) anotada a mano. Es lo que decide si una predicción cuenta como acierto (True Positive) o como falsa alarma (False Positive). Va de 0, sin solapamiento, a 1, cajas idénticas.

En la figura, la caja predicha está en rojo y la de verdad en verde. La IoU divide el área donde se superponen entre el área total que cubren las dos juntas.

$$IOU = \frac{\text{area of overlap}}{\text{area of union}} = \frac{\square}{\square}$$




IoU

**0,33**

Solapamiento parcial.

### Umbrales típicos de IoU

- **IoU ≥ 0,5:** el umbral clásico de PASCAL VOC para contar un acierto. COCO lo publica como AP50.
- **IoU ≥ 0,75:** más estricto, exige cajas casi exactas (AP75). La métrica principal de COCO promedia diez umbrales, de 0,50 a 0,95.
- **IoU por debajo del umbral:** la detección cuenta como False Positive (FP).

### ¿Por qué es importante?

- Sin IoU no habría forma de separar los aciertos (TP) de las falsas alarmas (FP).
- Es la base de las métricas que vienen después: precisión, recall, Average Precision (AP) y mean Average Precision (mAP).
- Mide la exactitud espacial de cada detección, además de si acierta la clase.

IoU vale 1 cuando ambas cajas coinciden y 0 cuando no comparten área. Para decidir TP, FP y FN también se tiene en cuenta la clase, la correspondencia uno a uno y un protocolo de evaluación.

# Fundamentos: TP, FP, FN (y por qué no hay TN)

### TP (True positives)

Detecciones correctas: cajas predichas que coinciden con objetos reales (IoU > umbral).

### FP (False positives)

Falsas alarmas: cajas predichas donde no hay objeto real o con IoU por debajo del umbral.

### FN (False negatives)

Objetos perdidos: objetos reales que el modelo no detectó.

## Por qué NO existen True Negatives (TN) en detección de objetos

La métrica de True Negatives, común en problemas de clasificación binaria, no aplica de la misma manera en la detección de objetos:

- **En clasificación binaria:** TN = «predice que no hay gato y efectivamente no hay gato».
- **En detección de objetos:** el modelo predice cajas delimitadoras en ubicaciones específicas donde cree que hay un objeto.
- No existe el concepto explícito de «predecir ausencia de objeto en una ubicación».
- Los sitios donde no se predice nada son infinitos: no se pueden contar.

Por esta razón, las métricas en detección de objetos se enfocan únicamente en **TP, FP y FN**, que son los únicos resultados que se pueden contar.

# Métricas de evaluación: precisión y recall

Saber si el modelo ha detectado algo no basta. La precisión dice cuánto puedes fiarte de lo que detecta; el recall, cuánto de lo que hay consigue encontrar.

### La importancia de la precisión

Mide la **fiabilidad de las detecciones**: con alta precisión hay pocos falsos positivos (detectar algo que no está o confundir su clase). Importa cuando una falsa alarma sale cara, como en inspección de calidad o en sistemas de seguridad.

$$\text{Precisión} = \frac{TP}{TP + FP}$$

Responde a: «De todas mis predicciones, ¿cuántas son correctas?».

### La importancia del recall

Mide la **capacidad de encontrar todos los objetos**: con alto recall hay pocos falsos negativos (objetos presentes que no se detectan). Importa cuando perder uno es grave, como una persona en vigilancia o un tumor en imagen médica.

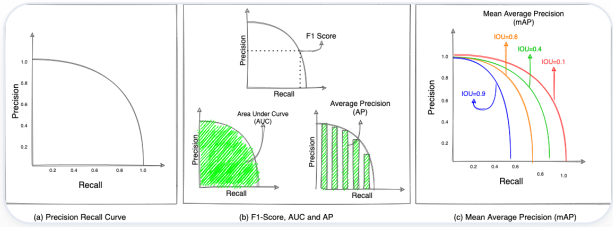
$$\text{Recall} = \frac{TP}{TP + FN}$$

Responde a: «De todos los objetos reales, ¿cuántos he detectado?».

**Cómo se leen:**

- **Precisión 0,9:** el 90 % de las detecciones son correctas. Puedes fiarte de lo que dice haber encontrado.
- **Recall 0,8:** ha encontrado el 80 % de los objetos que había. Se le escapa uno de cada cinco.

Las dos suelen tirar en direcciones opuestas, un **trade-off**, aunque la precisión no siempre sube al filtrar. Un modelo que detecta con mucha manga ancha para no perderse nada (recall alto) también da más falsas alarmas (precisión baja). El punto de equilibrio lo decide la aplicación.



# Calcula precisión y recall

Aplica las dos fórmulas a un recuento concreto.

## Hay 3 TP, 1 FP y 2 FN. ¿Cuáles son precisión y recall?

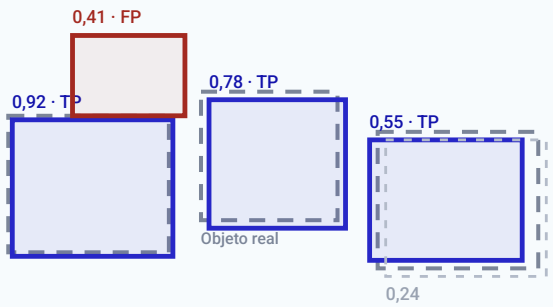
- A Precisión 0,60; recall 0,75.
- B Precisión 0,75; recall 0,75.
- C Precisión 0,75; recall 0,60.

Respuesta C. Exacto: 3/(3+1) y 3/(3+2).

Con 3 TP, 1 FP y 2 FN: precisión = 3 / (3 + 1) = 0,75 y recall = 3 / (3 + 2) = 0,60. La precisión mira el denominador de las predicciones; el recall, el de los objetos reales.

# Subir el umbral cambia el equilibrio

Al filtrar por puntuación el recall solo puede bajar; la precisión suele subir, aunque no siempre.




Precisión

**0,75**

Recall

**1,00**

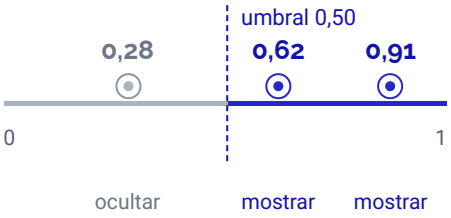
TP **3** FP **1** FN **0**

Se ven todos los objetos, pero también sobran predicciones.

Esta página recoge el estado con umbral 0,30. El emparejamiento sigue el protocolo habitual: las predicciones se ordenan por puntuación y cada una reclama el objeto libre con el que comparte más área, siempre que el IoU llegue a 0,50. La predicción de 0,24 solapa con el tercer objeto, pero llega tarde: ese objeto ya está emparejado, así que cuenta como falso positivo duplicado. Al subir el umbral desaparecen antes las predicciones dudosas: sube la precisión y baja el recall. La curva precisión-recall recorre ese intercambio umbral a umbral.

# Filtrar no cambia las cajas

El umbral de confianza decide qué predicciones se muestran; IoU evalúa su solapamiento.




El filtro cambia qué predicciones ves, no sus coordenadas.

**Subes el umbral de confianza sin cambiar las predicciones candidatas. ¿Qué pasa con las cajas que se muestran?**

- A Se conservan las mismas o menos cajas.
- B Aparecen categorías nuevas.
- C Aumenta necesariamente la IoU de todas las cajas restantes.

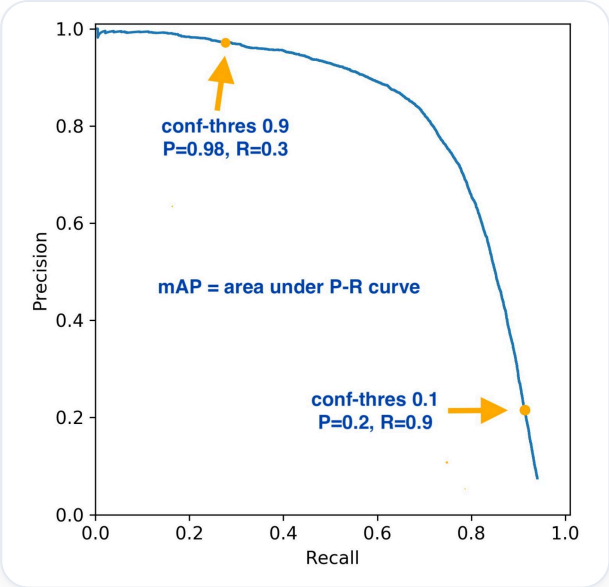
Respuesta A. Exacto: el filtro elimina las puntuaciones que quedan por debajo del nuevo umbral.

Al subir el umbral suelen desaparecer predicciones de baja puntuación: pueden bajar los falsos positivos y también aumentar los falsos negativos. La curva precisión-recall estudia ese intercambio recorriendo muchos umbrales.

# Métricas de evaluación: AP y mAP

**AP (Average Precision):** es el área bajo la curva precisión-recall. Resume en un número lo bien que el modelo encuentra los objetos (recall alto) sin llenarse de falsos positivos (precisión alta). Cuanto más alta, mejor.

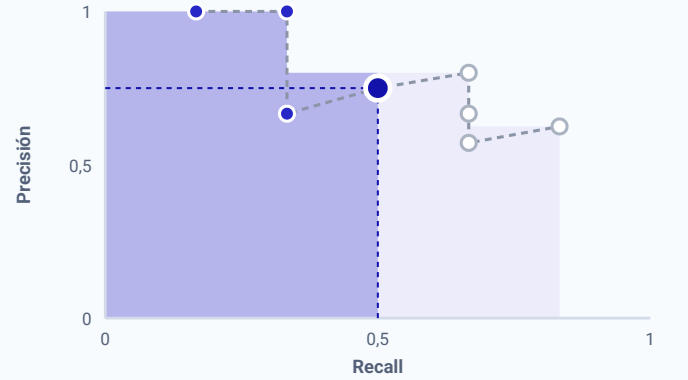
**mAP (mean Average Precision):** la métrica estándar para comparar detectores. Es la media de la AP de todas las clases del conjunto de evaluación. mAP@0.5 la calcula con IoU 0,5; mAP@[0.5:0.95], la métrica principal de COCO, la promedia además entre diez umbrales de 0,50 a 0,95.




Recorrer el umbral de confianza traza la curva: con umbral 0,9 la precisión es alta y el recall bajo; con 0,1 ocurre lo contrario. AP50 y AP promediada entre varios umbrales de IoU responden a protocolos distintos y no son intercambiables.

# La curva se dibuja bajando el umbral

Ocho detecciones de una clase, ordenadas por confianza, para seis objetos reales. Cada vez que el umbral deja pasar una más, la curva gana un punto; la AP es el área bajo su escalera.




| 0,95 | 0,90 | 0,82 | 0,76 | 0,64 | 0,51 | 0,43 | 0,30 |
|---|---|---|---|---|---|---|---|
| TP | TP | FP | TP | TP | FP | FP | TP |

Precisión

**0,75**

Recall

**0,50**

Área acumulada **0,47**

AP final **0,70**

Con umbral 0,76 entra un acierto: el recall sube a 0,50 y la precisión queda en 0,75.

Esta página recoge el estado con umbral 0,76: cuatro detecciones aceptadas, tres aciertos y un falso positivo, con precisión 0,75 y recall 0,50. Cada falso positivo baja la precisión sin mover el recall; cada acierto sube el recall. Para calcular el área, cada punto toma la mejor precisión que se alcanza a ese recall o a uno mayor, y por eso la curva sombreada es una escalera. Con las ocho detecciones el recall se queda en 0,83 porque un objeto no lo encontró ninguna, y la AP vale 0,70. COCO usa 101 puntos de recall en vez de todos, con el mismo principio.

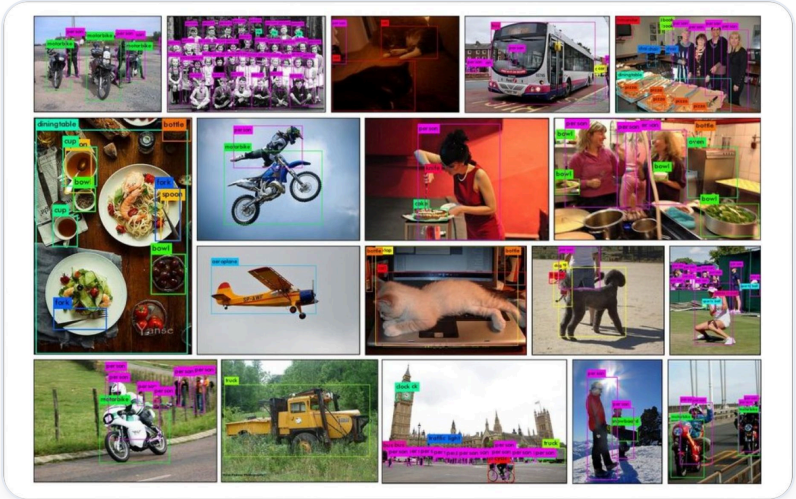
# Límites prácticos: COCO y el mundo real

El dataset COCO, con sus 80 clases estándar (persona, coche, perro, silla…), es excelente para benchmarking pero normalmente **no cubre las necesidades reales de industria o dominios especializados**.

**Problema del dominio:** una fábrica necesita detectar «tornillo oxidado», «conector suelto», «clip roto» o «soldadura defectuosa». Estos objetos específicos no existen en COCO.

**Coste de adaptación:** reanotar miles de imágenes para nuevas categorías es caro y lento. Entrenar desde cero o fine-tunear requiere tiempo de GPU, experimentación y validación.

Esta limitación motiva la necesidad de **métodos más flexibles**: detectores guiados por texto (zero-shot), modelos fundacionales multimodales, y técnicas que puedan adaptarse rápidamente a nuevos dominios sin anotación exhaustiva.



# El vocabulario depende de los datos

Un checkpoint aprende las clases de sus datos. Tu aplicación puede necesitar otras.

**Tu checkpoint solo conoce clases COCO y necesitas detectar un defecto industrial nuevo.**

- A Bajar mucho el umbral para crear la categoría.
- B Evaluar adaptación con datos etiquetados o un modelo adecuado al nuevo vocabulario.
- C Aumentar el tamaño de las cajas existentes.

Respuesta B. Exacto: la nueva categoría exige una estrategia y una evaluación en el dominio.

Un detector de vocabulario cerrado solo puede nombrar las categorías con las que fue entrenado. Ampliarlo exige anotar y reentrenar, o cambiar a un enfoque guiado por texto.

# Buenas prácticas al evaluar detectores

Evaluar un detector va más allá de mirar su mAP en COCO. Para desplegarlo de verdad hay que:

**1. Medir en tu dominio específico:** crea un conjunto de validación representativo de tus datos reales. Un modelo con 60 % mAP en COCO puede tener 40 % o 80 % en tu caso de uso particular.

**2. Equilibrar velocidad y precisión:** un detector con 2 % más de AP pero un 50 % más lento puede no compensar si necesitas procesar 30 FPS. Mide la latencia de extremo a extremo, no la cifra teórica del paper.

**3. Analizar errores cualitativamente:** inspecciona visualmente los casos de:

- **Falsos positivos (FP):** detecciones donde no hay objeto.
- **Falsos negativos (FN):** objetos que el modelo no detectó.
- **Casos límite:** objetos parcialmente ocluidos, muy pequeños, o en condiciones de iluminación extrema.

Los patrones de error te dirán si necesitas más datos, data augmentation específico, o cambiar de arquitectura.

# Mide donde vas a usarlo

Un benchmark no sustituye datos representativos, latencia medida y análisis de errores.

## Un detector logra mejor AP en un benchmark publicado. ¿Basta para elegirlo?

- A Sí, cualquier mejora de AP garantiza mejor producción.
- B Sí, si es el modelo más reciente.
- C No: hay que medir calidad y latencia con datos y condiciones representativos.

Respuesta C. Exacto: la elección depende del problema, del entorno y de los errores que aceptas.

Un AP mejor en un benchmark público indica rendimiento sobre ese conjunto y ese protocolo. La decisión necesita además datos de tu dominio, latencia medida en el hardware de destino y revisión de los errores que importan.

# El siguiente paso

Los detectores de este tema comparten tres límites. Los modelos multimodales del tema 3 atacan los tres.

### Limitaciones actuales

- Solo 80 clases COCO
- No entienden lenguaje
- Requieren reentrenamiento

### Modelos multimodales

- Entienden texto + imagen
- Zero-shot con lenguaje natural
- Sin reentrenamiento constante

# Detectar objetos en seis líneas

Lo mínimo que necesitas para tener cajas sobre una imagen tuya.

```
from rfdetr import RFDETRMedium
from PIL import Image

model = RFDETRMedium()
image = Image.open("mi_imagen.jpg").convert("RGB")

detections = model.predict(image, threshold=0.5)
print(detections.class_id, detections.confidence, detections.xyxy)
```

- **threshold** es el umbral de confianza: súbelo si ves falsas alarmas, bájalo si se pierden objetos.
- **xyxy** son las coordenadas de cada caja en píxeles, en el orden x mínimo, y mínimo, x máximo, y máximo.
- La primera ejecución descarga los pesos. En Colab conviene llamar antes a `model.optimize_for_inference()`.

RF-DETR se publica bajo Apache 2.0 y devuelve las clases de COCO. Para un vocabulario propio hace falta fine-tuning con un dataset anotado, cuyo coste estima la calculadora de presupuesto, tres pantallas más adelante.

# ¿De dónde han salido esos pesos?

La receta anterior baja un modelo ya entrenado antes de detectar nada. Ese paso silencioso es el tema siguiente.

### Por el paquete de Roboflow

Es lo que acabas de ejecutar. `rfdetr` trae el modelo, la descarga y el `predict` en una sola pieza, y es también por donde se reentrena con tus propias imágenes.

### Por el Hub de Hugging Face

El mismo RF-DETR está publicado como `Roboflow/rf-detr-medium` con licencia Apache 2.0, así que `transformers` lo carga igual que a cualquier otro modelo del curso.

```
from transformers import AutoImageProcessor, RfDetrForObjectDetection

processor = AutoImageProcessor.from_pretrained("Roboflow/rf-detr-medium")
model = RfDetrForObjectDetection.from_pretrained("Roboflow/rf-detr-medium")
```

Ese identificador de dos partes, **quién publica** y **qué publica**, es la unidad con la que se trabaja a partir de aquí. El tema 2 va de leer lo que hay detrás: qué licencia trae, cuánta memoria pide y cómo elegir entre los cientos que dicen hacer lo mismo.

Las dos vías cargan los mismos pesos y se diferencian en el envoltorio. El paquete `rfdetr` expone la API de Roboflow, con `predict` y el entrenamiento propio; `transformers` expone la interfaz común de la librería, que es la que comparten todos los modelos de los temas siguientes y la que permite intercambiar un detector por otro sin reescribir el resto. El identificador y el nombre de clase están tomados de la ficha del modelo, consultada el 17 de septiembre de 2026. El curso pasado RF-DETR solo se podía usar por el paquete; la integración en transformers es reciente.

# La misma familia hace tres cosas

El paquete que acabas de usar trae ahora segmentación y puntos clave. Cambia la clase y cambia lo que recibes, con la misma foto y las mismas tres líneas.

```
from rfdetr import RFDETRMedium

modelo = RFDETRMedium()
salida = modelo.predict("foto.jpg", threshold=0.5)

print(salida.xyxy)        # una caja por objeto
print(salida.class_id)    # su clase de COCO
```

Una caja por objeto, en píxeles. Sirve para contar, para disparar un aviso y para recortar la zona antes de mirarla con más detalle.

### El ejercicio

Haz una foto a tu mesa, con una persona dentro si puedes, y pasa las tres. Luego responde con cada salida a la misma pregunta: ¿cuánto ocupa ese objeto? La caja da un rectángulo, la máscara da los píxeles de verdad y el esqueleto no responde, porque mide otra cosa.

### Para pintarlo

Las tres salidas son objetos de `supervision`, así que se dibujan con `sv.BoxAnnotator`, `sv.MaskAnnotator` y `sv.EdgeAnnotator` sin tocar nada más.

Las tres tareas comparten el backbone DINOv2 y solo cambia la cabeza que va encima. Es la idea de modelo fundacional que recorre el curso, en tres líneas de código.

Segmentación y puntos clave se publican como «preview», es decir, acceso temprano pensado sobre todo para ajustar el modelo con un dataset propio; las clases son `RFDETRSegPreview` y `RFDETRKeypointPreview`, y la licencia del paquete abierto es Apache 2.0. Los puntos clave son los 17 de COCO, los mismos que se evalúan con OKS en el tema 6. Las variantes de detección aparecen en el tema 1, la segmentación es lo que se estudia en el tema 5 con SAM y la pose, en el tema 6: esta pantalla adelanta que el mismo paquete cubre las tres, con la diferencia de que aquí las clases son las de COCO y con SAM se segmenta cualquier cosa sin lista de clases. Consultado el 17 de septiembre de 2026 en la documentación de Roboflow.

# ¿Cuánto cuesta anotar tu dataset?

La cuenta que decide si fine-tuneas o te apoyas primero en un detector guiado por texto.

Clases que necesitas **4**

Imágenes por clase **300**

Segundos por imagen **45**

Imágenes que anotar

**1200**

Horas, con revisión incluida

**20**

Jornadas de una persona

**3,3**

Coste asumible para un conjunto propio. Aun así, empieza validando con zero-shot para saber si hace falta.

Estimación de orden de magnitud. Suma un 30 % de revisión sobre el tiempo de anotación y cuenta 6 horas productivas por jornada.

La página recoge el caso de 4 clases, 300 imágenes por clase y 45 segundos por imagen. Las cifras que más se disparan son las clases y los segundos por imagen: anotar segmentación o puntos clave multiplica el tiempo respecto a poner cajas. Antes de asumir ese coste conviene comprobar cuánto resuelve un detector guiado por texto sin anotar nada, que se ve en el tema 3.

# ¿Qué error te sale más caro?

El umbral no se elige en abstracto: depende de qué duele más en tu aplicación.

**¿Para qué usas el detector?**

- (●) Seguridad: detectar a una persona en zona restringida
- ( ) Inspección: encontrar piezas defectuosas antes de enviarlas
- ( ) Inventario: contar producto en una estantería
- ( ) Analítica: estimar flujo de gente en una calle

### Prioriza el recall

No detectar a alguien que sí está es el fallo grave. Una falsa alarma cuesta una comprobación; un objeto perdido cuesta el incidente.

**Qué hacer:** baja el umbral de confianza y asume más falsos positivos. Añade una segunda comprobación, humana o automática, para filtrarlos.

### Prioriza el recall, con revisión humana

Dejar pasar una pieza defectuosa llega al cliente; marcar una buena solo cuesta una revisión en planta.

**Qué hacer:** umbral bajo y una estación de revisión. Mide cuántas piezas buenas estás parando, porque ese es el coste que notará producción.

### Prioriza la precisión

Contar de más es peor que contar de menos: dispara reposiciones y pedidos equivocados que nadie revisa.

**Qué hacer:** sube el umbral hasta que apenas haya duplicados y vigila el recuento frente a un conteo manual de control.

### Busca el equilibrio y mide la tendencia

Aquí no importa cada persona concreta sino que el sesgo sea estable. Un error constante se corrige; uno que cambia con la hora o el clima, no.

**Qué hacer:** deja el umbral intermedio y valida sobre grabaciones de momentos distintos, no sobre una sola.

La regla general: si el coste de perder un objeto supera al de revisar una falsa alarma, baja el umbral; si es al revés, súbelo. Y en todos los casos, mide el efecto sobre tus propios datos, no sobre el benchmark.

# Antes de poner un detector en producción

Ocho comprobaciones que evitan casi todos los sustos.

- [ ] **Conjunto de validación propio**
  Imágenes de tu dominio, tus cámaras y tus condiciones de luz. No basta el mAP del benchmark.
- [ ] **Umbral elegido con criterio**
  Decidido según qué error cuesta más en tu caso, no dejado en el valor por defecto.
- [ ] **Latencia medida en el dispositivo real**
  End to end, incluyendo carga, preprocesado y posprocesado. No el número del paper.
- [ ] **Revisión visual de los errores**
  Mira falsos positivos, falsos negativos y casos límite: ocluidos, pequeños, contraluz.
- [ ] **Licencia comprobada**
  Del repositorio y de los pesos, en la versión concreta que vas a desplegar.
- [ ] **Versión y revisión fijadas**
  Anota el identificador del modelo y su revisión para poder reproducir el resultado.
- [ ] **Comportamiento ante entradas raras**
  Imagen vacía, muy oscura, resolución distinta, formato inesperado.
- [ ] **Plan de seguimiento**
  Cómo vas a detectar que el rendimiento cae cuando cambien las condiciones.

# Ahora, con tu propia imagen

Prueba RF-DETR en tu copia del cuaderno.

1. Guarda una copia en Drive.
2. Ejecuta el detector con una imagen.
3. Cambia el umbral de confianza: **0,25 · 0,50 · 0,75**.

Un detector cerrado reconoce sus clases; el tema 3 añade lenguaje y zero-shot.

**Abrir en Colab ↗**

En Colab: Archivo → Guardar una copia en Drive. Empieza por inferencia. La disponibilidad de GPU varía; el entrenamiento necesita además el dataset indicado en el notebook. La comparación entre 0,25, 0,50 y 0,75 permite observar qué predicciones desaparecen sin confundir confianza con IoU. El tema 3 amplía el vocabulario cerrado con modelos que relacionan visión y lenguaje; antes, el tema 2 enseña de dónde salen los pesos.